ISKOLAI PROJEKT / 2026. NOVEMBER

A programozási nyelvek evolúciója

FORTRAN → C → RUST

Három korszak. Három tervezési szemlélet. Egy közös programozási feladat.

1957 / FORTRAN	1972 / C	2015 / RUST
Számítás és kifejezőerő	Rendszerközeli kontroll	Biztonság és ownership

KÉSZÍTŐK

Milán · Gergő · Dominik

BEADÁS

2026. november 23. | 3 fős csoport

TARTALOM Történelmi kontextus • technikai elemzés • C–Rust összehasonlítás • futtatható példák • konklúzió és hivatkozások

01 / BEVEZETÉS

Mit jelent a nyelvek fejlődése?

A programozási nyelvek nem egyszerűen „régiből újba” fejlődnek. Mindegyik a saját kora problémáira reagál: a gépi kód nehézkességére, a rendszerszoftverek hordozhatóságára vagy az alacsony szintű programok memóriabiztonsági kockázataira.

Kutatási kérdés

Hogyan változott a programozási nyelvek tervezésének hangsúlya 1957 és 2015 között, és milyen kompromisszumok maradtak fontosak a mai fejlesztésben?

Három mérföldkő

Év / korszak	Nyelv	Megoldandó probléma
1957 – úttörők	Fortran	Hatékony, olvasható tudományos számítások.
1972 – alapozók	C	Operációs rendszerekhez alkalmas, hordozhatóbb rendszernyelv.
2015 – modern	Rust	Gépközeli teljesítmény memóriabiztonsági garanciákkal.

Vizsgálati módszer

- Elsődleges történeti és hivatalos nyelvdokumentációk összevetése [1–8].
- Azonos számítás elkészítése mindhárom nyelven: öt egész szám összege és átlaga.
- Paradigma, típusrendszer, memóriafelelősség, szintaxis és mai szerep összehasonlítása.

FONTOS KÜLÖNBSÉG A nyelvekhez rendelt évszám a történelmi mérföldkő, a mintakód viszont mai fordítóval használható nyelvváltozatban készült. Fortran esetén Fortran 2008, C esetén C11, Rust esetén 2021 edition.

02 / TÖRTÉNELEM ÉS TECHNIKA

FORTRAN / a számítás nyelve

A Fortran (FORmula TRANslation) fejlesztése 1954-ben indult az IBM-nél, a John Backus vezette csapattal. 1957-es megjelenése megmutatta, hogy a mérnökök és kutatók a gépi kódhoz képest közvetlenebbül írhatnak le matematikai számításokat. A hangsúly nem a grafikus felület, hanem a numerikus problémák hatékony kifejezése volt. [1–2]

Mi volt az eredeti cél?

A matematikai képletek és ciklusok gépbarát, de ember által is olvasható leírása. A korai fordítóknak különösen fontos volt a gyors végrehajtás, hiszen a számítási kapacitás drága erőforrásnak számított.

Technikai jellemzők

- Paradigma: történetileg imperatív/procedurális; a modern Fortran további struktúrákat, modulokat és objektumorientált elemeket is kínál.
- Típusosság: statikus. Régi jellegzetessége az implicit típusmegállapítás; modern kódban az `implicit none` segít elkerülni a véletlen változókat.
- Memória: statikus és automatikus adatok mellett `allocatable` tömbök és `allocate` / `deallocate`. Bizonyos hatókörökben automatikus felszabadítás is megvalósul. [7–8]
- Erősségek: tudományos és mérnöki számítások, tömbműveletek, numerikus könyvtárak.

Részlet a projektkódból

```
allocate(scores(5))
scores = [12, 18, 25, 31, 44]
total = sum(scores)
average = real(total, real64) / real(size(scores), real64)
deallocate(scores)
```

ÖRÖKSÉG A Fortran nem „kihalt őskövület”: a tudományos és mérnöki számításokban ma is használják. A példában viszont modern Fortran-szintaxis szerepel, nem az 1957-es FORTRAN I. [7]

03 / TÖRTÉNELEM ÉS TECHNIKA

C / a rendszerprogramozás alapja

Dennis Ritchie a Bell Labs környezetében alakította ki a C nyelvet a korai Unix rendszer fejlesztéséhez. A fejlesztés 1969–1973 között zajlott, kiemelt időszaka 1972 volt. A C a B és BCPL előzményeire épített, de típusokat és olyan eszközöket adott, amelyekkel gépközeli, mégis strukturált szoftver készíthető. [3]

Milyen problémát oldott meg?

A rendszerszoftvereknél fontos az előre jelezhető erőforrás-kezelés és a hardver közvetlen elérése. A C a gépi kódtól magasabb absztrakciós szintet adott, miközben a memóriára és a teljesítményre továbbra is nagy ráhatást engedett.

Technikai jellemzők

- Paradigma: elsősorban procedurális, strukturált; függvényekre és adatstruktúrákra épít.
- Típusosság: statikus, de a pointerek, implicit/explicit konverziók és castok miatt gyengébb védelmi korlátokkal, mint a Rust. A „gyenge típusú” címke önmagában félrevezető lehet.
- Memória: automatikus lokális adatok mellett dinamikus foglalás `malloc`-kal, felszabadítás `free`-vel. Ezek használata fegyelmet igényel. [9–10]
- Erősségek: beágyazott rendszerek, operációsrendszer-közeli komponensek, C ABI, rendszerszintű könyvtárak.

Részlet a projektkódból

```
int *scores = malloc(count * sizeof *scores);
if (scores == NULL) return EXIT_FAILURE;
/* ... a tomb elemeinek feldolgozasa ... */
free(scores);
```

KOCKÁZAT A `free` elmulasztása memóriaszivárgáshoz vezethet; a már felszabadított pointer használata vagy a túlindexelés meghatározatlan viselkedést okozhat. A példaprogram ellenőrzi a foglalás sikerét. [9–10]

04 / TÖRTÉNELEM ÉS TECHNIKA

RUST / biztonság fordítás közben

A Rust megalkotásának célja, hogy a rendszerszintű programozásban együtt jelenjen meg a nagy teljesítmény, az alacsony szintű kontroll és a jobb memóriabiztonság. A stabil Rust 1.0 verzió 2015. május 15-én jelent meg; a nyelv fejlesztése ennél korábban kezdődött. [4]

Ownership és borrowing

A Rust ownership modelljében egy értékhez meghatározott tulajdonos tartozik. A kölcsönzés (`&T`, `&mut T`) lehetővé teszi az adatok átadását anélkül, hogy azokat minden alkalommal átmásolnánk vagy a tulajdonjogot átruháznánk. A fordító számos élettartam- és aliasolási hibát már fordítás előtt elutasít. [5–6]

Technikai jellemzők

- Paradigma: többparadigmás; imperatív elemeket, erős típusrendszert, iterátorokat és funkcionális mintákat vegyít.
- Típusosság: statikus, erős típusellenőrzéssel és típusinferencia-támogatással.
- Memória: ownership, borrowing, RAII-jellegű `Drop`; normál, biztonságos Rust kódhoz nem szükséges hagyományos garbage collector.
- Korlátok: meredekebb tanulási görbe, a fordítói megszorítások tudatos tervezést igényelnek; `unsafe` esetén külön felelősség terheli a fejlesztőt.

Részlet a projektkódból

```
fn calculate(scores: &[i32]) -> (i32, f64) {
    let total: i32 = scores.iter().sum();
    let average = f64::from(total) / scores.len() as f64;
    (total, average)
}
```

MIÉRT FONTOS? A `&[i32]` egy kölcsönzött szelet: a `calculate` használhatja az adatokat, de nem lesz a `Vec` tulajdonosa. [5–6]

05 / ÖSSZEHASONLÍTÁS

Három szemlélet ugyanarra a problémára

Szempont	Fortran	C	Rust
Mérföldkő	1957	1972 körül	1.0: 2015
Fő cél	Numerikus számítás	Rendszerprogramozás	Biztonságos rendszerek
Paradigma	Procedurális + modern bővítések	Procedurális	Többparadigmás
Típusrendszer	Statikus; implicit none ajánlott	Statikus; rugalmas konverziók	Statikus; szigorú ellenőrzés
Dinamikus memória	ALLOCATE / DEALLOCATE	malloc / free	Vec, ownership, Drop
Referencia/ adatátadás	Tömbök, eljárási argumentumok	Pointerek	Borrowing, szeletek
Tipikus környezet	HPC, tudomány	OS, firmware, könyvtárak	Rendszerek, eszközök
Fő kihívás	Régi és új kód együttélése	Memóriabiztonság	Tanulási görbe

Közvetlen C–Rust összevetés

A C a memória címét és élettartamát gyakran a programozóra bízza; a Rust a biztonságos hivatkozások használatát a típusrendszerrel és a borrow checkerrel korlátozza. A C szabadsága akkor előny, ha közvetlen hardverkezelés vagy interoperabilitás szükséges; a Rust akkor vonzó, ha a memóriahibák jelentős részét már fordításkor szeretnénk kizárni. [3,5,9–10]

NEM VERSENYTESZT Az összehasonlítás nem állítja, hogy a Rust minden programban gyorsabb, vagy hogy a C szükségszerűen hibás. A három megoldás erőforrás-kezelési és olvashatósági döntéseit mutatja be.

06 / KÓDPÉLDÁK / 1

Azonos feladat: Fortran és C

Feladat: az [12, 18, 25, 31, 44] egész számok összegének és átlagának kiszámítása. A kódok dinamikus tárolással szemléltetik a memóriakezelési modellt.

FORTRAN 2008 – SRC/FORTRAN/MAIN.F90

```
program statistics_fortran
    use, intrinsic :: iso_fortran_env, only : real64
    implicit none
    integer, allocatable :: scores(:)
    integer :: total
    real(real64) :: average

    allocate(scores(5))
    scores = [12, 18, 25, 31, 44]
    total = sum(scores)
    average = real(total, real64) / real(size(scores), real64)

    write(*, '(A,I0)') 'Sum: ', total
    write(*, '(A,F0.2)') 'Average: ', average

    deallocate(scores)
end program statistics_fortran
```

C11 – SRC/C/MAIN.C

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    const int input[] = {12, 18, 25, 31, 44};
    const size_t count = sizeof input / sizeof input[0];
    int *scores = malloc(count * sizeof *scores);
    if (scores == NULL) {
        fputs("Allocation failed\n", stderr);
        return EXIT_FAILURE;
    }

    int total = 0;
    for (size_t i = 0; i < count; ++i) {
        scores[i] = input[i];
        total += scores[i];
    }
    const double average = (double)total / (double)count;

    printf("Sum: %d\n", total);
    printf("Average: %.2f\n", average);

    free(scores);
    return EXIT_SUCCESS;
}
```

07 / KÓDPÉLDÁK / 2

Rust és ellenőrizhető eredmények

RUST 2021 EDITION – SRC/RUST/SRC/MAIN.RS

```
// Rust: Vec owns its elements; calculate borrows an immutable slice.
fn calculate(scores: &[i32]) -> (i32, f64) {
    let total: i32 = scores.iter().sum();
    let average = f64::from(total) / scores.len() as f64;
    (total, average)
}

fn main() {
    let scores = vec![12, 18, 25, 31, 44];
    let (total, average) = calculate(&scores);

    println!("Sum: {total}");
    println!("Average: {average:.2}");
    // The Vec's allocation is freed when scores goes out of scope.
}
```

Várt kimenet mindhárom nyelven

```
Sum: 130
Average: 26.00
```

Fordítás és ellenőrzés

```
bash scripts/check.sh
# Rust kulon: cargo run --manifest-path src/rust/Cargo.toml
```

A C és Fortran példák a projekt készítésekor GCC, illetve GNU Fortran fordítóval sikeresen lefordultak és pontosan a fenti kimenetet adták. Rust-fordító ebben a készítői környezetben nem állt rendelkezésre, ezért a Rust futtatását a beadás előtt Cargo-val külön ellenőrizni kell.

KÖVETKEZTETÉS A matematikai eredmény mindhárom esetben azonos. A különbség a memória tulajdonlásának, a felszabadításnak és az adatátadásnak a kódbeli kifejezésében rejlik.

08 / KONKLÚZIÓ ÉS ÖRÖKSÉG

Mi maradt meg az egyes korszakokból?

Fortran: a matematikai probléma a középpontban

A korai Fortran a magas szintű fordítás életképességét demonstrálta a tudományos világ számára. A mai Fortran továbbra is hasznos a numerikus ökoszisztémában, ahol sok évtized munkája és optimalizált algoritmusa halmozódott fel. Az eredeti cél lényegében fennmaradt, miközben a nyelv struktúrái modernizálódtak. [1–2,7]

C: a szintaxis és az interfészek tartós hatása

A C szemlélete és kapcsos zárójelre épülő szintaxisa számos későbbi nyelvet befolyásolt. Ökoszisztémájának fontos része a C-könyvtárakhoz kapcsolódó bináris interfész. A nyelv ma is releváns alacsony szintű komponensekhez, de a memóriahibák kockázata tartós fejlesztési kihívás. [3,9–10]

Rust: nagyobb szerepet kap a megelőzés

A Rust nem egyszerűen új szintaxist adott: több erőforrás- és élettartamszabályt fordításidejű ellenőrzéssé alakított. A cél nem az összes programhiba felszámolása, hanem meghatározott biztonsági hibakategóriák korai kiszűrése. [4–6]

A projekt három fő tanulsága

01 A nyelvtervezés a problémakörnyezetre reagál: számítás → hordozható rendszerprogramozás → memóriabiztonság.

02 A „modern” nem feltétlenül váltja le a „régi”-t: a különböző nyelvek ma is különböző területeken hasznosak.

03 A fő kompromisszum sokszor nem a kód hossza, hanem a biztonság, a kontroll és a fejlesztői felelősség eloszlása.

09 / MÓDSZERTAN ÉS FORRÁSOK

Hivatkozások és csoportmunka

A kutatás a kiadott feladatlap követelményei alapján készült. Az alábbi forrásokban szereplő adatok szolgálnak a történeti és nyelvi állítások alapjául. Teljes, kattintható forráslista: `resources/Forrasok.md`. Megtekintés: 2026.10.19.

[1] IBM Fortran: https://www.ibm.com/history/fortran

[2] IBM John Backus: https://www.ibm.com/history/john-backus

[3] D. M. Ritchie The Development of the C Language: https://www.nokia.com/bell-labs/about/dennis-m-ritchie/chist.html

[4] Rust Blog Announcing Rust 1.0: https://blog.rust-lang.org/2015/05/15/Rust-1.0/

[5] Rust Book Ownership: https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html

[6] Rust Book Borrowing: https://doc.rust-lang.org/book/ch04-02-references-and-borrowing.html

[7] GNU GNU Fortran: https://www.gnu.org/software/gcc/fortran/

[8] Intel Fortran Dynamic Allocation: intel.com/.../dynamic-allocation.html

[9] cppreference C malloc: https://en.cppreference.com/c/memory/malloc

[10] cppreference C free: https://en.cppreference.com/c/memory/free

Javasolt csapatfelosztás

Fő	Feladat
Milán	Kódok, Git-repozitórium, C–Rust technikai összehasonlítás
Gergő	Történelmi kutatás, Fortran- és C-háttér
Dominik	Összehasonlító táblázatok, konklúzió, prezentáció

A tényleges munkamegosztást és a készítők nevét beadás előtt egyeztessétek. A közös Git-repozitóriumba feltöltés a csoport feladata.